

Interface Prototype

1. Purpose

このドキュメントは、IRCサーバ実装における担当者間Interfaceを定義する。

本プロジェクトでは複数人で並行実装するため、以下を明確にする。

- 各担当が外部へ提供するクラス
- 外部から呼び出してよい関数
- 引数と戻り値
- 担当者間の依存関係
- Network / IO層とProtocol / Command層の接続方法

全体設計・責務分割は `dev_docs/design.md` に記載する。

実装状況・変更予定・未実装項目は `dev_docs/roadmap.md` に記載する。

2. Dependency Policy

2.1 基本方針

本プロジェクトでは、担当者間の依存を最小化する。

特に重要な方針は以下。

- A: Network / IO層は、IRCコマンドの意味を知らない。
- B: Protocol / Command層は、Network / IO層の内部実装を知らない。
- Bは `Server` , `Connection` , `Poller` , `ConnectionManager` に依存しない。
- Bは送信処理を直接行わない。
- Bはコマンド処理結果を `CommandResult` として返す。
- A / Server は `CommandResult` を受け取り、対象fdのsend bufferへ積む。
- C1はClientとServerStateを管理する。
- C2はChannelとChannelModesを管理する。
- ChannelはClientを所有しない。 `Client*` を参照するだけ。
- Clientの生成・削除はC1が担当する。

3. Processing Flow

基本的な処理フローは以下。

```

A / Server
  poll() でfdイベントを受け取る
  ↓
A / Connection
  recv bufferから complete line を取り出す
  ↓
B / Parser
  line を Message に変換する
  ↓
B / CommandDispatcher
  Message と ServerState を使ってコマンド処理する
  ↓
B / CommandDispatcher
  CommandResult を返す
  ↓
A / Server
  CommandResult 内の OutgoingMessage を対象fdのsend bufferへ積む
  ↓
A / Connection
  POLLOUTでsend bufferを送信する

```

4. Common Interface Types

4.1 OutgoingMessage

送信対象fdと送信文字列を表す。

```

struct OutgoingMessage {
    int      fd;
    std::string message;

    OutgoingMessage(int targetFd, const std::string& text);
};

```

役割

- Bが「誰に何を送るか」を表現する。
- 実際の `send()` は行わない。
- A / Server がこの情報を見てsend bufferへ積む。

4.2 CommandResult

1つのIRCコマンド処理結果を表す。

```

struct CommandResult {
    std::vector<OutgoingMessage> replies;
    bool shouldDisconnect;

    CommandResult();

    void addReply(int fd, const std::string& message);
};

```

役割

- コマンド処理後に送るべきメッセージを保持する。
- 必要に応じて切断要求を表す。
- BからAへ返される。
- BがAの `Connection` や `Server` を直接触らないための境界になる。

5. A: Network / IO Interface

担当: A

Aはfd、socket、poll、recv buffer、send bufferを管理する。

AはIRCコマンドの意味を知らない。

5.1 Server

外部に提供する主な関数

関数プロトタイプ	戻り値	利用者	内容
<code>void run();</code>	<code>void</code>	<code>main</code>	サーバのメインループを開始する
<code>void queueSend(int fd, const std::string& msg);</code>	<code>void</code>	<code>Server</code> 内部	指定fdのsend bufferへ送信文字列を積む
<code>void applyCommandResult(const CommandResult& result);</code>	<code>void</code>	<code>Server</code> 内部	<code>CommandResult</code> を処理し、返信をsend bufferへ積む
<code>void disconnectClient(int fd);</code>	<code>void</code>	<code>Server</code> 内部	対象fdの接続を安全に切断する

補足

`CommandDispatcher` は `Server` を直接呼ばない。
`Server` が `CommandResult` を受け取り、`queueSend()` を呼ぶ。

5.2 Connection

外部に提供する主な関数

関数プロトタイプ	戻り値	利用者	内容
<code>int getFd() const;</code>	<code>int</code>	<code>Server</code>	<code>Connection</code> が持つfdを返す
<code>bool readFromSocket();</code>	<code>bool</code>	<code>Server</code>	<code>recv()</code> してrecv bufferへ追加する。切断・致命エラーならfalse
<code>bool writeToSocket();</code>	<code>bool</code>	<code>Server</code>	send bufferから送れる分だけ <code>send()</code> する。切断・致命エラーならfalse
<code>bool hasCompleteLine() const;</code>	<code>bool</code>	<code>Server</code>	<code>\r\n</code> 単位の完全な1行があるか確認する
<code>std::string popLine();</code>	<code>std::string</code>	<code>Server</code>	recv bufferから1行取り出す
<code>void queueSend(const std::string& msg);</code>	<code>void</code>	<code>Server</code>	send bufferへ送信文字列を追加する
<code>bool hasPendingOutput() const;</code>	<code>bool</code>	<code>Server</code>	send bufferに未送信データがあるか確認する

補足

`Connection` はIRCコマンドの意味を知らない。

やらないこと:

- `PASS` 判定
- `NICK` 判定
- `JOIN` 処理
- `PRIVMSG` 配送先判定
- `MODE` 解釈

5.3 Poller optional

Poller は必要に応じて Server から分離する。

分離する場合の主な関数

関数プロトタイプ	戻り値	利用者	内容
void add(int fd, short events);	void	Server	fdをpoll監視対象に追加する
void remove(int fd);	void	Server	fdをpoll監視対象から削除する
void enableWrite(int fd);	void	Server	POLLOUT 監視を有効化する
void disableWrite(int fd);	void	Server	POLLOUT 監視を無効化する
int wait();	int	Server	poll() を呼ぶ
const std::vector<struct pollfd>& fds() const;	const std::vector<struct pollfd>&	Server	pollfd一覧を参照する

重要ルール

実際に poll() を呼ぶ場所は1箇所に限定する。

Poller を分離する場合も、Server から Poller::wait() を呼ぶ構造にする。
各 Connection が個別に poll() / select() を呼ぶことは禁止する。

5.4 ConnectionManager optional

ConnectionManager は必要に応じて Server から分離する。

分離する場合の主な関数

関数プロトタイプ	戻り値	利用者	内容
void addConnection(int fd);	void	Server	fdからConnectionを作成する
void removeConnection(int fd);	void	Server	Connectionを削除する
Connection* getConnection(int fd);	Connection*	Server	fdからConnectionを取得する
bool hasConnection(int fd) const;	bool	Server	指定fdのConnectionが存在するか確認する
void queueSend(int fd, const std::string& msg);	void	Server	指定fdのConnectionのsend bufferへ積む
void clear();	void	Server	全Connectionを解放する

補足

Bは ConnectionManager を直接呼ばない。
Bは CommandResult を返し、A / Server が送信処理を行う。

6. B: Protocol / Command Interface

担当: B

BはIRCメッセージの解析、コマンド振り分け、返信生成を担当する。

BはAのNetwork / IOクラスに依存しない。

6.1 Message

外部に提供する主な関数

関数プロトタイプ	戻り値	利用者	
<code>const std::string& getCommand() const;</code>	<code>const std::string&</code>	CommandDispatcher	command
<code>const std::vector<std::string>& getParams() const;</code>	<code>const std::vector<std::string>&</code>	CommandDispatcher	parameter
<code>size_t getParamCount() const;</code>	<code>size_t</code>	CommandDispatcher	parameter
<code>const std::string& getSingleParam(size_t index) const;</code>	<code>const std::string&</code>	CommandDispatcher	指定index
<code>bool hasParam(size_t index) const;</code>	<code>bool</code>	CommandDispatcher	指定index

想定構造

```
class Message {
private:
    std::string      _command;
    std::vector<std::string> _params;
};
```

補足

`trailing` は `_params` の最後の要素として扱う。

例:

```
PRIVMSG #room :hello world

command = "PRIVMSG"
params  = ["#room", "hello world"]
```

6.2 Parser

外部に提供する主な関数

関数プロトタイプ	戻り値	利用者	内容
<code>static Message parse(const std::string& line);</code>	Message	Server	1行文字列をMessageへ変換する

補足

`Parser` はrecv bufferを扱わない。
recv bufferからcomplete lineを切り出すのはA / Connectionの責務。

6.3 CommandDispatcher

外部に提供する主な関数

関数プロトタイプ	戻り値	利用者	内容
<code>CommandResult dispatch(int fd, const Message& msg, ServerState& state);</code>	CommandResult	Server	IRC commandを実行し、結果を返す

補足

`dispatch()` は送信処理を直接行わない。

やらないこと:

- `send()`
- `queueSend()`
- `Connection` 操作
- `Poller` 操作
- `Server` 操作

やること:

- `Message` のcommandを見る
- `ServerState` から `Client` / `Channel` を取得する
- 必要に応じて状態を更新する
- `ReplyBuilder` で返信文字列を作る
- `CommandResult` に送信対象と文字列を詰める

6.4 ReplyBuilder

外部に提供する主な関数

関数プロトタイプ
<code>static std::string welcome(const Client& client);</code>
<code>static std::string needMoreParams(const Client& client, const std::string& command);</code>
<code>static std::string alreadyRegistered(const Client& client);</code>
<code>static std::string passwordMismatch();</code>
<code>static std::string nickInUse(const std::string& nick);</code>
<code>static std::string noSuchNick(const Client& client, const std::string& nick);</code>
<code>static std::string noSuchChannel(const Client& client, const std::string& channel);</code>
<code>static std::string userNotInChannel(const Client& client, const std::string& nick, const std::string& channel);</code>
<code>static std::string notOnChannel(const Client& client, const std::string& channel);</code>
<code>static std::string notRegistered();</code>
<code>static std::string chanOpPrivsNeeded(const Client& client, const std::string& channel);</code>
<code>static std::string channelIsFull(const Client& client, const std::string& channel);</code>
<code>static std::string inviteOnlyChan(const Client& client, const std::string& channel);</code>
<code>static std::string badChannelKey(const Client& client, const std::string& channel);</code>
<code>static std::string join(const Client& client, const std::string& channel);</code>
<code>static std::string privmsg(const Client& from, const std::string& target, const std::string& text);</code>
<code>static std::string kick(const Client& actor, const Client& target, const std::string& channel, const std::string& reason);</code>
<code>static std::string invite(const Client& actor, const Client& target, const std::string& channel);</code>
<code>static std::string topic(const Client& client, const std::string& channel, const std::string& topic);</code>
<code>static std::string topicReply(const Client& client, const std::string& channel, const std::string& topic);</code>
<code>static std::string noTopic(const Client& client, const std::string& channel);</code>
<code>static std::string mode(const Client& client, const std::string& channel, const std::string& modeLine);</code>

関数プロトタイプ

```
static std::string channelModeIs(const Client& client, const std::string& channel, const std::string& modeLine);
```

7. C1: Client / ServerState Interface

担当: C1

C1はClientの状態、Clientの生死、サーバ全体の辞書を管理する。

7.1 Client

外部に提供する主な関数

関数プロトタイプ	戻り値	利用者	内容
int getFd() const;	int	B / C2	Clientに対応するfdを取得する
const std::string& getNick() const;	const std::string&	B / C2	nickを取得する
const std::string& getUsername() const;	const std::string&	B	usernameを取得する
const std::string& getRealname() const;	const std::string&	B	realnameを取得する
const std::string& getHost() const;	const std::string&	B	hostを取得する
std::string getFullPrefix() const;	std::string	B	nick!user@host 形式を返す (RFC 1459 Section 2.3 準拠)
void setUsername(const std::string& username);	void	B	usernameを設定する
void setRealname(const std::string& realname);	void	B	realnameを設定する
void setHost(const std::string& host);	void	A	hostを設定する (接続時)
void setPassOk(bool ok);	void	B	PASS成功状態を設定する
bool isPassOk() const;	bool	B	PASS済みか確認する
bool isRegistered() const;	bool	B	登録完了済みか確認する
bool canRegister() const;	bool	B	PASS / NICK / USER が揃ったか確認する
void markRegistered();	void	B	登録完了状態にする

注意

Client::setNick() は外部から直接呼ばない。

nick変更は必ず ServerState::updateNick() を通す。

getFullPrefix() について

サーバーがクライアントにメッセージを送る際、送信元を nick!user@host 形式で付与する必要がある (RFC 1459 Section 2.3)。詳細は dev_docs/rfc1459_prefix_analysis.md 参照。

7.2 ServerState

外部に提供する主な関数

関数プロトタイプ	戻り値	利用者	内容
const std::string& password() const;	const std::string&	B	サーバpasswordを取得

関数プロトタイプ	戻り値	利用者	内容
<code>void addClient(int fd);</code>	<code>void</code>	Server	新規接続時にClientを作
<code>void removeClient(int fd);</code>	<code>void</code>	Server / B	切断時にClientを削除す
<code>void removeClientFromAllChannels(Client& client);</code>	<code>void</code>	ServerState内部 / B	全ChannelからClient参
<code>Client* getClientByFd(int fd);</code>	<code>Client*</code>	B	fdからClientを取得する
<code>Client* getClientByNick(const std::string& nick);</code>	<code>Client*</code>	B	nickからClientを取得す
<code>bool nickExists(const std::string& nick) const;</code>	<code>bool</code>	B	nick重複を確認する
<code>void updateNick(Client& client, const std::string& newNick);</code>	<code>void</code>	B	Clientのnickとnick辞書
<code>Channel* getChannel(const std::string& name);</code>	<code>Channel*</code>	B	Channelを取得する。な
<code>Channel* getOrCreateChannel(const std::string& name);</code>	<code>Channel*</code>	B	Channelを取得する。な
<code>void removeChannelIfEmpty(const std::string& name);</code>	<code>void</code>	B / Server	空Channelを削除する

削除ルール

`removeClient(int fd)` はClientを削除する前に `removeClientFromAllChannels()` 相当の処理を必ず行う。

削除対象:

- Channel member
- Channel operator
- invited list

その後、空になったChannelを削除し、`fd -> Client` と `nick -> Client` の辞書を更新する。

7.3 ClientRegistry optional

`ClientRegistry` は `ServerState` が肥大化した場合に分離する。

初期実装では必須ではない。

分離する場合の主な関数

関数プロトタイプ	戻り値	利用者	内容
<code>void add(int fd);</code>	<code>void</code>	<code>ServerState</code>	Clientを追加する
<code>void remove(int fd);</code>	<code>void</code>	<code>ServerState</code>	Clientを削除する
<code>Client* findByFd(int fd);</code>	<code>Client*</code>	<code>ServerState</code>	fdからClientを取得する
<code>Client* findByNick(const std::string& nick);</code>	<code>Client*</code>	<code>ServerState</code>	nickからClientを取得する
<code>bool nickExists(const std::string& nick) const;</code>	<code>bool</code>	<code>ServerState</code>	nick重複を確認する
<code>void updateNick(Client& client, const std::string& newNick);</code>	<code>void</code>	<code>ServerState</code>	nick辞書を更新する

8. C2: Channel Interface

担当: C2

C2はChannel内部状態とChannelModesを管理する。

8.1 Channel

外部に提供する主な関数

関数プロトタイプ	戻り値	利用者	内容
<code>const std::string& name() const;</code>	<code>const std::string&</code>	B / C1	channel名を取得する
<code>bool hasMember(Client* client) const;</code>	<code>bool</code>	B	Clientが参加済みか確認する
<code>void addMember(Client* client);</code>	<code>void</code>	B	memberを追加する
<code>void removeMember(Client* client);</code>	<code>void</code>	B	memberを削除する
<code>void removeClient(Client* client);</code>	<code>void</code>	B / C1	member / operator / invited list からまとめて削除する
<code>std::vector<Client*> members() const;</code>	<code>std::vector<Client*></code>	B	member一覧を取得する
<code>size_t memberCount() const;</code>	<code>size_t</code>	B	member数を取得する
<code>bool isOperator(Client* client) const;</code>	<code>bool</code>	B	Clientがchannel operatorか確認する
<code>void addOperator(Client* client);</code>	<code>void</code>	B	operator権限を付与する
<code>void removeOperator(Client* client);</code>	<code>void</code>	B	operator権限を剥奪する
<code>void invite(Client* client);</code>	<code>void</code>	B	招待リストに追加する
<code>bool isInvited(Client* client) const;</code>	<code>bool</code>	B	招待済みか確認する
<code>void removeInvite(Client* client);</code>	<code>void</code>	B	招待状態を解除する
<code>void setTopic(const std::string& topic);</code>	<code>void</code>	B	topicを設定する
<code>const std::string& topic() const;</code>	<code>const std::string&</code>	B	topicを取得する
<code>ChannelModes& modes();</code>	<code>ChannelModes&</code>	B	mode変更用
<code>const ChannelModes& modes() const;</code>	<code>const ChannelModes&</code>	B	mode参照用
<code>bool isEmpty() const;</code>	<code>bool</code>	B / C1	memberが0人か確認する

補足

`Channel` は `Client` を所有しない。
`Channel` は `Client*` を保持するだけで、`delete` はしない。

JOIN時のoperator初期化

新規作成されたChannelに最初のClientが参加する場合、参加処理は `addMember()` に加えて `addOperator()` も行う。

この判定は `CommandDispatcher` または `ChannelService::join()` が行う。

8.2 ChannelModes

外部に提供する主な関数

関数プロトタイプ	戻り値	利用者	内容
<code>bool inviteOnly() const;</code>	<code>bool</code>	B	<code>+i</code> 状態を取得する
<code>bool topicRestricted() const;</code>	<code>bool</code>	B	<code>+t</code> 状態を取得する
<code>bool hasKey() const;</code>	<code>bool</code>	B	<code>+k</code> 状態を取得する
<code>const std::string& key() const;</code>	<code>const std::string&</code>	B	channel keyを取得する
<code>int limit() const;</code>	<code>int</code>	B	user limitを取得する。無制限なら <code>-1</code>

関数プロトタイプ	戻り値	利用者	内容
<code>void setInviteOnly(bool value);</code>	<code>void</code>	B	<code>+i</code> / <code>-i</code> を設定する
<code>void setTopicRestricted(bool value);</code>	<code>void</code>	B	<code>+t</code> / <code>-t</code> を設定する
<code>void setKey(const std::string& key);</code>	<code>void</code>	B	<code>+k</code> を設定する
<code>void unsetKey();</code>	<code>void</code>	B	<code>-k</code> を設定する
<code>void setLimit(int limit);</code>	<code>void</code>	B	<code>+l</code> を設定する
<code>void unsetLimit();</code>	<code>void</code>	B	<code>-l</code> を設定する

8.3 ChannelService optional

ChannelService は CommandDispatcher が肥大化した場合に分離する。

初期実装では必須ではない。

分離する場合の主な関数

関数プロトタイプ	戻り値	利用者	内容
<code>static bool canJoin(Channel& channel, Client& client, const std::string& key);</code>	<code>bool</code>	B	JOIN可能か判定する
<code>static void join(Channel& channel, Client& client);</code>	<code>void</code>	B	ClientをChannelに参加させる 最初のmemberならclient
<code>static bool canKick(Channel& channel, Client& actor);</code>	<code>bool</code>	B	KICK権限があるか判定
<code>static void kick(Channel& channel, Client& target);</code>	<code>void</code>	B	ClientをChannelからkickする
<code>static bool canInvite(Channel& channel, Client& actor);</code>	<code>bool</code>	B	INVITE権限があるか判定
<code>static void invite(Channel& channel, Client& target);</code>	<code>void</code>	B	Clientを招待する
<code>static bool canChangeTopic(Channel& channel, Client& actor);</code>	<code>bool</code>	B	TOPIC変更権限があるか判定
<code>static void setTopic(Channel& channel, const std::string& topic);</code>	<code>void</code>	B	topicを変更する
<code>static bool canChangeMode(Channel& channel, Client& actor);</code>	<code>bool</code>	B	MODE変更権限があるか判定
<code>static bool isFull(Channel& channel);</code>	<code>bool</code>	B	user limitに達しているか判定
<code>static std::string currentModeLine(const Channel& channel);</code>	<code>std::string</code>	B	MODE照会用のmode line

9. Important Rules

9.1 B does not depend on A

CommandDispatcher は以下をincludeしない。

- Server.hpp
- Connection.hpp
- Poller.hpp
- ConnectionManager.hpp

Bは CommandResult を返すだけにする。

9.2 A applies CommandResult

A / Server は CommandResult を受け取って送信処理を行う。

```
CommandResult result = dispatcher.dispatch(fd, msg, state);
applyCommandResult(result);
```

`applyCommandResult()` は、`result.replies` を見て各fdのsend bufferへ積む。

9.3 Nick update

nick変更は必ず `ServerState::updateNick()` を通す。

NG:

```
client.setNick(newNick);
```

OK:

```
state.updateNick(client, newNick);
```

9.4 Client ownership

Clientの生成・削除はC1が担当する。

- `ServerState::addClient(fd)` で作成する。
 - `ServerState::removeClient(fd)` で削除する。
 - `Channel` は `Client*` を参照するだけ。
 - `Channel` は `Client*` を `delete` しない。
-

9.5 Channel operator

operator権限は `Client` ではなく `Channel` が管理する。

NG:

```
client.setOperator(true);
```

OK:

```
channel.addOperator(&client);
```

理由:

1人のClientは複数Channelに参加でき、Channelごとにoperator権限が異なるため。

9.6 POLLOUT

送信すべきデータがあるfdだけ `POLLOUT` を有効にする。

- send bufferが空でない場合、`POLLOUT` を有効にする。
 - send bufferが空になった場合、`POLLOUT` を無効にする。
 - 常時 `POLLOUT` を監視し続けない。
-

9.7 Client removal cleanup

`ServerState::removeClient(fd)` はClientを削除する前に、全Channelから該当Clientへの参照を除去する。

削除対象:

- member
- operator
- invited list

`Channel` は `Client*` を所有しないため、Client削除後にChannel内へ `Client*` を残してはいけない。

9.8 TOPIC / MODE read operations

`TOPIC #channel` と `MODE #channel` は状態変更ではなく照会として扱う。

- `TOPIC #channel` はtopicがあれば `topicReply()`、なければ `noTopic()` を返す。
 - `MODE #channel` は `currentModeLine()` で現在のmode文字列を作り、`channelModeIs()` を返す。
-

10. Open Questions

以下は実装しながら確定する。

10.1 Poller を最初から分離するか

現時点では optional。

A担当の既存モックを活かし、まずは `Server` 内で管理してもよい。

肥大化した場合に `Poller` へ分離する。

10.2 ConnectionManager を最初から分離するか

現時点では optional。

まずは `Server` が `fd -> Connection` を直接管理してもよい。

send buffer管理が膨らむ場合、`ConnectionManager` へ分離する。

10.3 ClientRegistry を分離するか

現時点では optional。

まずは `ServerState` が `fd -> Client` と `nick -> Client` を直接持つ。

肥大化した場合に `ClientRegistry` へ分離する。

10.4 ChannelService を分離するか

現時点では optional。

まずは `CommandDispatcher` が `Channel` / `ChannelModes` を直接操作する。

肥大化した場合に `ChannelService` へ分離する。